DRAFT Fractional Vertex Cover Online on Trees

Bartosz Bromblik

Abstract

1 Introduction

In classic vertex cover problem each vertex of the graph is either included in *the set* or not, which can also be expressed as giving each vertex a *value* of 1 or 0, respectively. Then, problem requirements are set as $|v| + |u| \geq 1$ for each edge (u, v). In fractional covering we use a less strict requirement, namely that each value is a real number in range $[0, 1]$. Inequalities are stated alike.

We define *score* of a fractional covering as the *sum of values of all vertices in the current covering* divided by the *sum of values all vertices in on optimal covering*. Optimal fractional vertex cover on trees can be found in the same way as the classic one (correctness proof is also the same). Score, defined as such, is always ≥ 1. A score is *better* if it's smaller (closer to optimal). A covering is *better* if it has a better score.

We imagine *online* covering problem as a turn-based game of two players: algorithm and adversary. The former changes values in vertices to meet problem requirements while also keeping the score as good as possible. The latter builds the graph by adding one edge at a time, trying to make the solution (covering) worse.

If the algorithm can arbitrarly change values of vertices, it can always achieve a score of 1. But if we require monotonicity, ie. values never get decreased, the whole score minimalisation problem becomes interesting.

2 Linear programming

In this scenario we play as the adversary and we try to maximise the score achieved by **any** algorithm on the given input.

We start with a list of edges (of length T) we want to add in each turn and try to encode that as an input to some linear programming solver. Keep in mind that even though the algorithm knows the whole tree (and all tricks used) in advance, it still has to assert that all requirements are met after each turn.

2.1 Formulation

General Let $v_i^{(t)}$ stand for the value of i-th vertex after t-th turn. List of things we put into our linear programming problem (for a given list of edges):

1. Declare *score*, which we want (the algorithm) to minimise.
2. Define limits on all values. For all vertices i, $v_i^{(0)} \geq 0$ and $v_i^{(T)} \leq 1$.
3. Force monotonicity of all values. For each vertex i and each turn t (from 1 to T) $v_i^{(t-1)} \leq v_i^{(t)}$.
4. Add edges. When we want to connect vertices i and j in turn t, we write $v_i^{(t)} + v_j^{(t)} \geq 1$.
5. Compare with optimal. After each turn t we add $\sum_i v_i^{(t)} \leq Opt^{(t)} \cdot score$, where $Opt^{(t)}$ stands for the sum of values of all vertices in the optimal covering.

Symmetry Breaking It is alway beneficial for the adversary to add the second edge in each connected component to the vertex with lower value, because it forces the algorithm to increase the total sum by more (for edge with values $a \leq b$, connecting the third vertex to the first one results in the grand total of $1 + b$, which is greater or equal to $1 + a$, which we get by doing the other move).

Therefore, when we see a second edge (v_a, v_c) begin added to a connected component $\{v_a, v_b\}$ (at time t), we write that $v_a^{(t)} \leq v_b^{(t)}$. We call that *Symmetry Breaking*, abbreviated as *SB*.

Branching We allow ourselves to consider and encode multiple ways of building the tree. More precisely, at turn t we add some edge in branch p and another one in branch q. Keep in mind that branches cannot be merged.

One can think of them as an option for the adversary to change the tactic depending on the state of the tree. Or just an another possibility the algorithm needs to account for when assigning values.

To denote multiple branches we can use $v_i^{(t,b)}$ where b is the index of branch that vertex belongs to (in turn t). Multiple branches can come from the same state, but they are completely independent, so we can encode on them one by one, using just their indexes b.

As in step 3., we need to assert monotonicity of values. If branch q forks from branch p at turn t, we write $v_i^{(t-1,p)} \leq v_i^{(t,q)}$ for each vertex i.

Also, we need to terminate every branch, similarly to step 2., so we write $v_i^{(T_b,b)} \leq 1$ for each vertex, where T_b is the last turn in branch b.

It's worth noting, that a complete solution would consider all possible branches at all turns, but this causes an exponential growth of the size of the linear programming problem. Our goal here is to achieve the highest possible score with an affordable amount of computation.

Attacks An attack on vertex v is a way to force a value of 1 in this vertex. It is achieved by connecting an extra node v' to v with an edge, so that $|v| + |v'| \geq 1$.

We usually attack multiple vertices at once. More precisely, for a correct *normal* vertex cover S of the current tree, we attack all of the vertices in S. We encode that similarly to step 5., namely $|S| + \sum_{v_i \notin S} v_i^{(t)} \leq |S| \cdot score$. This may be done for all S, after each turn.

This approach can be further simplified to only using correct coverings that are inlcusion-wise minimal. For S and S', $S \subset S'$, $S' \setminus S = v_s$, if $|S| + \sum_{v_i \notin S} v_i^{(t)} = A \leq |S| \cdot score$, then also $A - v_s^{(t)} + 1 \leq A + 1 \leq |S| \cdot score + 1 \leq |S| \cdot score + score = |S'| \cdot score$. Therefore adding an inequality (step 5.) for any $|S'|$ will not change the *score*. Moreover, not all inlcusion-wise minimal $|S|$ are guaranteed to affect the score, and one can choose to omit them.

Attacks can be concider as a simplified version of branching.

2.2 Results

During testing we found that a good adversary strategy, both in terms of score and computation time, is a line made in a very specific order.

Start with one edge. We say it has a left and right end. Next, add an edge on the right, breaking the symmetry. Later, add one edge on the right, one on the left. The, one on the left and one on the right. And so on.

We do not use branching. The only attacks used are for the optimal coverings - when no two connected vertices are in the set.

With such means we achieved a score of over **1.69**, using 23 edges (as described above).

Key observations:

1. Value of each vertex gets changed at most twice - once for each edge attached to it. These are the only two moments when a new requirement using the value of that vertex is added. Extra changes could not help.

2. After the first turn, both vertices get assigned the same value of $\frac{1}{2}$.

We chose to investigate and solve the linear programming problem by hand, as the number of edges goes to infinity, which we describe in the next section.

3 Vertex Cover lower bound

Description Instead of focusing on the *score* as in $|S| + \sum_{v_i \notin S} v_i^{(t)} \leq |S| \cdot score$, we use $\alpha = score - 1$. Then we ask about $\sum_{v_i \notin S} v_i^{(t)} \leq |S| \cdot \alpha$.

We will only consider inequalities (as in 2.2) after even number of turns, which is a less strict requirement, but sufficient for our analysis.

For the first three vertices v_0, v_1 and v_2 with edges (v_0, v_1) in turn 1 and (v_1, v_2) in turn 2 we assign variables:

- $x := |v_0^{(1)}|$
- $y := |v_1^{(1)}|$, $Y := |v_1^{(2)}|$
- $z := |v_2^{(2)}|$

Notice that we use lowercase for the vertices with one edge and uppercase for the vertices with two edges.

After the first two edges, all vertices are added in *pairs* - one vertex on one end, one vertex on the other end, and only then do we add an inequality. Therefore, we name their values symmetrically: a_i^L for the left one and a_i^R for the right one. i is the index of that pair. a_i^L, a_i^R denote value after adding one edge to the vertex and A_i^L, A_i^R after adding the second edge, respectively.

3.1 Calculations

Base There are two kinds of inequalities, for edges and for vertex coverings. The former are marked green and the latter in blue.

- First we have $x + y \geq 1$, and from Symmetry Breaking we get $x \geq y$. Therefore $x \geq \frac{1}{2}$.
- From $x + z \leq \alpha$ and $Y + z \geq 1$ we get $z \leq \alpha - \frac{1}{2}$ and $Y \geq \frac{3}{2} - \alpha$.
- Next we have $Z + a_0^R \geq 1$, $X + a_0^L \geq 1$, and $a_0^L + Y + a_0^R \leq 2 \cdot \alpha$.

Let $a_i = \frac{1}{2} \cdot (a_i^L + a_i^R)$, as there is really no difference between those two. Alike, $A_i = \frac{1}{2} \cdot (A_i^L + A_i^R)$.

- With new notation we get $Y + 2 \cdot a_0 \leq 2 \cdot \alpha$. Therefore $a_0 \leq \frac{3}{2} \cdot \alpha - \frac{3}{4}$.
- Adding inequalities together we get $X + Z + 2 \cdot a_0 \geq 2$, so $X + Z \geq \frac{7}{2} - 3 \cdot \alpha$.
- From $a_1^L + X + Z + a_1^R = X + Z + 2 \cdot a_1 \leq 3 \cdot \alpha$ we conclude $a_1 \leq 3 \cdot \alpha - \frac{7}{4}$

Recursion Notice that in blue inequalities there are always two variables written in lowercase - values of vertices at the ends of the line, and *some* other variables. To simplify, we name their sum S_i. So we have:

- $2 \cdot a_0 + S_0 \leq 2 \cdot \alpha$, where $S_0 = Y \geq \frac{3}{2} - \alpha$
- $2 \cdot a_1 + S_1 \leq 3 \cdot \alpha$, where $S_1 = X + Z \geq \frac{7}{2} - 3 \cdot \alpha$
- $2 \cdot a_2 + S_2 \leq 4 \cdot \alpha$, where $S_2 = 2 \cdot A_0 + Y = 2 \cdot A_0 + S_0 \geq 7 - 7 \cdot \alpha$
- $2 \cdot a_3 + S_3 \leq 5 \cdot \alpha$, where $S_3 = 2 \cdot A_1 + X + Z = 2 \cdot A_1 + S_1$

We can clearly see two patterns: $2 \cdot a_i + S_i \leq (i + 2) \cdot \alpha$ and $S_{i+2} = S_i + 2 \cdot A_i$. Because $a_{i+1}^L + A_i^L \geq 1$ and $a_{i+1}^R + A_i^R \geq 1$ we have $2 \cdot A_i \geq 2 - 2 \cdot a_{i+1}$. So $S_{i+2} \geq S_i + 2 - 2 \cdot a_{i+1}$. Using $2 \cdot a_{i+1} + S_{i+1} \leq (i + 3) \cdot \alpha$ we can further simplify it to:

$$S_{i+2} \geq S_{i+1} + S_i + 2 - (i + 3) \cdot \alpha$$

Induction For given S_1 and S_2 we can solve that recurrence relation and get:

$$S_i \geq F_{i-2} \cdot S_1 + F_{i-1} \cdot S_2 + 2 \cdot (F_i - 1) - (F_{i+2} + 3 \cdot F_i - i - 4) \cdot \alpha \qquad \text{for } i \geq 2$$

F_i stands for the fibonacci sequence, where $F_0 = 0$, $F_1 = 1$ and $F_{i+2} = F_{i+1} + F_i$ for $i \geq 0$. We can check by hand that the above is true for S_2 and S_3. For $i \geq 4$:

$$S_i \geq S_{i-1} + S_{i-2} + 2 - (i+1) \cdot \alpha$$

$$\begin{aligned} &\geq (F_{i-3} \cdot S_1 + F_{i-2} \cdot S_2 + 2 \cdot (F_{i-1} - 1) - (F_{i+1} + 3 \cdot F_{i-1} - i - 3) \cdot \alpha) \\ &+ (F_{i-4} \cdot S_1 + F_{i-3} \cdot S_2 + 2 \cdot (F_{i-2} - 1) - (F_i + 3 \cdot F_{i-2} - i - 2) \cdot \alpha) + 2 - (i+1) \cdot \alpha \end{aligned}$$

$$\begin{aligned} &= (F_{i-3} + F_{i-4}) \cdot S_1 + (F_{i-2} + F_{i-3}) \cdot S_2 + 2 \cdot (F_{i-1} - 1 + F_{i-2} - 1 + 1) \\ &\quad -\alpha \cdot (F_{i+1} + F_i + 3 \cdot F_{i-1} + 3 \cdot F_{i-2} - i - 3 - i - 2 + i + 1) \end{aligned}$$

$$= F_{i-2} \cdot S_1 + F_{i-1} \cdot S_2 + 2 \cdot (F_i - 1) - (F_{i+2} + 3 \cdot F_i - i - 4) \cdot \alpha$$

Therefore the inequality holds for all $i \geq 2$.

Solution We can substitute $S_1 \geq \frac{7}{2} - 3 \cdot \alpha$ and $S_2 \geq 7 - 7 \cdot \alpha$ and get:

$$S_i \geq F_{i-2} \cdot \frac{7}{2} - F_{i-2} \cdot 3 \cdot \alpha + F_{i-1} \cdot 7 - F_{i-1} \cdot 7 \cdot \alpha + 2 \cdot (F_i - 1) - (F_{i+2} + 3 \cdot F_i - i - 4) \cdot \alpha$$

$$= \left(\frac{7}{2} \cdot F_{i-2} + 7 \cdot F_{i-1} + 2 \cdot F_i - 2\right) - \alpha \left(3 \cdot F_{i-2} + 7 \cdot F_{i-1} + 3 \cdot F_i + F_{i+2} - i - 4\right)$$

$$= \left(\frac{7}{2} \cdot F_{i-1} + \frac{11}{2} \cdot F_i - 2\right) - \alpha \left(4 \cdot F_{i-1} + 6 \cdot F_i + F_{i+2} - i - 4\right)$$

$$= \left(2 \cdot F_i + \frac{7}{2} \cdot F_{i+1} - 2\right) - \alpha \left(2 \cdot F_i + 4 \cdot F_{i+1} + F_{i+2} - i - 4\right)$$

$$= \left(\frac{3}{2} \cdot F_{i+1} + 2F_{i+2} - 2\right) - \alpha \left(2 \cdot F_{i+1} + 3 \cdot F_{i+2} - i - 4\right)$$

Because $2 \cdot a_i + S_i \leq (i+2) \cdot \alpha$, $a_i \leq \frac{1}{2} \cdot ((i+2) \cdot \alpha - S_i)$:

$$a_i \leq \frac{1}{2} \left((i+2) \cdot \alpha + \alpha \left(2 \cdot F_{i+1} + 3 \cdot F_{i+2} - i - 4\right) - \left(\frac{3}{2} \cdot F_{i+1} + 2 \cdot F_{i+2} - 2\right)\right)$$

$$= \frac{1}{4} \left(\alpha \cdot (4 \cdot F_{i+1} + 6 \cdot F_{i+2} - 4) - (3 \cdot F_{i+1} + 4 \cdot F_{i+2} - 4)\right)$$

Bound As we required above, each variable has to be in range $[0, 1]$, including all a_i^L and a_i^R. If $a_i^L \geq 0$ and $a_i^R \geq 0$, then also $a_i = \frac{1}{2} \cdot (a_i^L + a_i^R) \geq 0$.

$$0 \leq a_i \leq \frac{1}{4} \left(\alpha \cdot (4 \cdot F_{i+1} + 6 \cdot F_{i+2} - 4) - (3 \cdot F_{i+1} + 4 \cdot F_{i+2} - 4)\right)$$

$$\implies \alpha \cdot (4 \cdot F_{i+1} + 6 \cdot F_{i+2} - 4) \geq (3 \cdot F_{i+1} + 4 \cdot F_{i+2} - 4)$$

$$\implies \alpha \geq \frac{3 \cdot F_{i+1} + 4 \cdot F_{i+2} - 4}{4 \cdot F_{i+1} + 6 \cdot F_{i+2} - 4}$$

Now we calculate the limit of that fraction, knowing that $\lim\limits_{i \to \infty} \frac{F_{i+1}}{F_i} = \phi = \frac{1+\sqrt{5}}{2}$:

$$\lim_{i \to \infty} \frac{3 \cdot F_{i+1} + 4 \cdot F_{i+2} - 4}{4 \cdot F_{i+1} + 6 \cdot F_{i+2} - 4} = \lim_{i \to \infty} \frac{3 + 4 \cdot \frac{F_{i+2}}{F_{i+1}} - \frac{4}{F_{i+1}}}{4 + 6 \cdot \frac{F_{i+2}}{F_{i+1}} - \frac{4}{F_{i+1}}}$$

$$= \frac{3 + 4 \cdot \phi}{4 + 6 \cdot \phi} = \frac{5 + 2 \cdot \sqrt{5}}{7 + 3 \cdot \sqrt{5}} = \frac{5 - \sqrt{5}}{4} = \frac{3 - \phi}{2}$$

In conclusion, we have just shown that the *score* of an infinite vertex cover game on a line (described earlier) is at least $1 + \alpha = \frac{5-\phi}{2} \approx 1.690983$